

MLSys 2026 Scheduling Contest – Track A

Toni Boehnlein Pál András Papp

April 24, 2026

1 Overview

We treat the scheduling problem as a two-level search: an outer *partition* of operations into subgraphs, and an inner *coupling* layer that stitches adjacent subgraphs together via fast-memory retention. Both levels optimise the same roofline cost model, share a thread-safe memoisation cache, and are driven by a Fiduccia–Mattheyses (FM) local search embedded inside an evolutionary multi-start loop. Symmetry in the input DAG (parallel branches, repeated series motifs) is detected once via Merkle hashing and used to guide local search and crossover.

The binary targets the 2–120s timeouts in the contest harness: the time budget is parsed from the benchmark file name and strictly honoured by a single deadline propagated to every worker thread.

2 Cost model

The reference cost is re-implemented bottom-up in `src/core/subgraph.cpp`. For a candidate op-set we derive the tile-slice sizes by backward propagation from the boundary outputs, classify each boundary tensor as *once/row/col/tile/stream load* (based on which tile-index component drives it), and then sum a per-tile roofline $\max(\text{compute}, \text{memory})$. Split- k is handled exactly: the first and last k -steps pay the full per-tile I/O and the output evict, while the middle $(nk-2)$ steps only stream inputs. Snake traversal (RowMajor / ColMajor) is implemented as closed-form loop counts, not a per-tile simulation, so enumerating every feasible (w, h, k, snake) config is cheap.

Feasibility is a first-class concept. `Subgraph::create` refuses any op-set that has mixed internal/external consumers of the same tensor, inconsistent boundary-output dimensions, or a disconnected sub-DAG. The PW-sink rule ($k=1$ whenever a Pointwise produces a boundary output) is baked into the precomputed candidate set for k . Tiling divisibility ($w \mid W_{\text{out}}$, etc.) is precomputed as a fast-path intersection. These invariants are verified by 47 unit/integration tests covering each move and each cost-model branch.

3 Partition search

The partition layer minimises the sum of subgraph costs assuming no retention. The state is a set of groups (op subsets); moves are MERGE, STEAL, RECOMPUTE, EJECT, INTERNAL-EJECT, SPLIT, and two tensor-level operators that work on shared inputs (Tensor-MERGE/Tensor-EXTRACT). Each move validates acyclicity on the condensed group DAG before evaluation (modified Kahn with OR-semantics for recomputed ops), and ephemeral-gap preservation (mixed-consumer constraint).

3.1 FM with drift

An FM pass forms an active set from a small random seed of ops and then iterates: pop the best move across the active set, apply it, lock the moved op so it cannot initiate again this pass, and expand the active set with the neighbouring ops of the affected groups (which are re-evaluated with the locked set in mind). We accept worsening moves up to a *drift budget* $\varepsilon \cdot C_0$ below the best snapshot seen so far; once the cumulative gain drops beyond this bound the pass aborts. The outer loop applies cosine-annealed temperature to the floor/drift fractions and a multiplicative *heat* that widens the search after each non-improving pass and narrows it after each improvement. On non-improving passes we also run a greedy-kick from the perturbed end state – a cheap way to land in an adjacent basin.

3.2 Evolutionary pool

Generation 0 seeds the pool from six structurally diverse initialisers (trivial, chain+edge, seed+grow, rev-topo, tensor-aligned, random) plus a symmetry-driven init that reuses detected parallel/series patterns. Each seed is refined by greedy + FM. Subsequent generations pick mutation (2/3) or crossover (1/3) tasks. Mutations are structural (random merges, splits, reassignments, block moves, tensor merges) and ignore cost, providing the diversity that the gain-guided FM cannot.

Crossover uses agreement clustering à la Sanders–Schulz: ops co-grouped in both parents form inherited clusters, the remainder is reassigned greedily. We additionally implement *DAG-cut* crossover which slices along topologically compatible boundaries and splices in a donor sub-partition, and a symmetry-aware variant that restricts donor candidates to Merkle-equivalent regions.

Pool admission is diversity-aware: partitions are compared via adjusted Rand index (ARI) on the contingency table, canonicalised by Merkle hashes so structurally symmetric partitions are collapsed. Insertion protects the best-cost entry, admits any partition that is either strictly better or sufficiently far from the nearest neighbour.

4 Symmetry detection

ML workloads are full of exact structural repetition – parallel attention heads, MoE experts, transformer blocks stacked in series – and exploiting that repetition is far cheaper than re-searching for it. We detect symmetry once up front and feed it back into initialisation, mutation, crossover, and pool diversity.

Merkle fingerprints. Each op gets a structural hash built by two directional passes over the DAG. The forward hash composes an op’s init hash (type, base cost, input/output tensor shapes) with the sorted forward hashes of its predecessors, iterated up to the longest-chain depth so information fully propagates. The backward hash does the same from successors. An op’s *combined* hash $fwd \oplus bwd$ uniquely identifies its global structural role: two ops share a combined hash iff their entire predecessor and successor cones are isomorphic, with matching tensor dimensions. Hashing iterates to a fixed point in $O(chain \cdot |ops|)$ time and is computed once at the start of the pipeline.

Parallel patterns. The forward-only hash is used to form *orbits* – sets of ops with identical predecessor cones. An orbit-level DAG is built with top/bottom topological-distance labels, and orbits are grown into connected *components* by a two-phase merger: Phase 1 merges only same-symmetry adjacent orbits (preserving the replication factor), Phase 2 greedily extends while allowing the symmetry to drop, as long as it stays > 1 . A level constraint (top and bottom ranges may differ by at most one) prevents creating cycles in the condensed super-node DAG. The output is a list of `SymmetricPatterns`, each reporting N isomorphic components with matching op lists.

Series patterns. A separate detector scans the topological order for consecutive blocks of K ops with identical internal structure and inter-block wiring (using the iteration-0 init hash so that chain-end effects don’t break the match). Transformer stacks, deep chains and MoE expert towers typically surface as one series pattern covering the bulk of the graph.

Consumption. Patterns feed four parts of the search: (i) a `symm_init` strategy that solves the partition on the representative component and replicates the grouping to the others, giving a high-quality seed for free on symmetric inputs; (ii) a coverage-weighted symmetric-mutation operator that applies a random partition edit to every replica simultaneously, so the child stays on the symmetric manifold; (iii) DAG-cut crossover, which restricts donor regions to Merkle-equivalent slices so the cut boundaries are known to match; (iv) pool-diversity canonicalisation, where `merkle_canonicalise` rank-matches ops within each equivalence class between two partitions before computing ARI, ensuring that two labellings that differ only by a symmetry permutation hash to distance 0 rather than a spurious positive.

5 Retention and coupling

Retention is modelled as a directed coupling edge $g_a \rightarrow g_b$ carrying a set of retained tensors. Each group has at most one incoming and one outgoing edge, so coupling chains are disjoint paths in the group DAG. The cost of a coupled pair recomputes both endpoints with retention context and caches the result in a second cost-cache tier (`evaluate_with_context`).

The coupled-partition layer runs its own FM/evo loop with coupling-aware moves (`COUPLE`, `DECOUPLE`, `REROUTE`, plus all partition moves lifted to respect chains). Phase 2 seeds the coupling pool from ordering heuristics (DFS retain-score and beam-search ordering); Phase 3 lets the coupled FM evo explore from there until the deadline. The final solution falls back to the uncoupled best partition if the coupled candidate fails validation.

6 Engineering notes

A two-tier shared cost cache memoises `eval_set` results keyed by the sorted op-set, with a second tier adding (entering, retain) for coupling; all workers share one instance and hit rates exceed 90% after the first generation. The partition maintains an op→group index and a compact `GroupDAG` so cycle checks and dirty-set propagation are $O(|E|)$ per move. Search is embarrassingly parallel at task granularity (one FM outer loop per task), using up to 8 threads with a mutex-guarded diversity pool. The budget is clamped to 95% of the posted timeout and propagated as an absolute deadline, so every FM pass and cost evaluation checks wall time and exits cleanly before the harness kills the process.

7 Results

On the five released benchmarks (`mlsys-2026-{1,5,9,13,17}`) the solver returns valid solutions within the allotted time. The evolutionary loop keeps improving the best partition on the larger instances up to the 60s deadline; on small instances the pool converges quickly and the remaining budget is spent on coupling refinement.